

LES METHODES D'EXTENSION

Le problème

En général, pour **ajouter des fonctionnalités à une classe**, nous pourrions **soit modifier le code source de la classe, soit créer une classe dérivée** de notre classe et ajouter ces fonctionnalités.

Dans ce chapitre nous allons voir qu'il existe un autre moyen pour **étendre une classe** grâce aux méthodes d'extension. Elles sont intéressantes si nous n'avons pas la main sur le code source de la classe ou si la classe n'est pas dérivable.

Partons à la découverte de ces fameuses méthodes...

C'est quoi une méthode d'extension ?

Comme son nom l'indique, une méthode d'extension permet d'étendre une classe en lui rajoutant des méthodes. Ceci est pratique lorsque **nous ne possédons pas le code source d'une classe** ou qu'il s'avère difficile de la modifier.

D'une manière générale, lorsque l'on souhaite modifier une classe dont on n'a pas le code source, on utilise une classe dérivée. Ceci est impossible avec des objets qui ne sont pas dérivables, comme c'est le cas pour les structures comme `int` ou `double`. Mais également comme c'est le cas pour la classe `String`. En effet, `String` n'est pas dérivable.

Si vous ne me croyez pas, vous pouvez toujours tenter de compiler le code suivant :

```
1public class StringEvoluee : String
2{
3}
```

Créer une méthode d'extension

Si par exemple nous souhaitons créer une méthode permettant de crypter une chaîne de caractères dans un format que nous seuls comprenons, il serait judicieux de créer une méthode dans une classe `StringCryptee` qui dérive de `string`.

Comme ceci n'est pas possible, la seule chose qui nous reste c'est de créer une méthode statique utilitaire faisant cet encodage :

```
1class Program
2{
3    static void Main(string[] args)
4    {
5        string chaineNormale = "Bonjour à tous";
6        string chaineCryptee = Encodage.Crypte(chaineNormale);
7        Console.WriteLine(chaineCryptee);
8        chaineNormale = Encodage.Decrypte(chaineCryptee);
9        Console.WriteLine(chaineNormale);
10    }
11
12}
13
14public static class Encodage
15{
16    public static string Crypte(string chaine)
17    {
18        return Convert.ToBase64String(Encoding.Default.GetBytes(chaine));
19    }
20}
```

```

21 public static string Decrypte(string chaine)
22 {
23     return Encoding.Default.GetString(Convert.FromBase64String(chaine));
24 }
25 }

```

*

Bon, ok, notre encodage secret ne l'est pas tant que ça.

Il s'avère que j'utilise ici un encodage en base 64, algorithme archi connu. Mais bon, c'est pour l'exemple.

Utiliser une méthode d'extension

Ces méthodes statiques jouent bien leur rôle. Mais il est possible de faire en sorte que ces deux méthodes deviennent des méthodes d'extensions de la classe `string`. Il suffit d'utiliser le mot-clé `this` devant le premier paramètre de la méthode afin de créer une méthode d'extension :

```

1 public static class Encodage
2 {
3     public static string Crypte(this string chaine)
4     {
5         return Convert.ToBase64String(Encoding.Default.GetBytes(chaine));
6     }
7
8     public static string Decrypte(this string chaine)
9     {
10        return Encoding.Default.GetString(Convert.FromBase64String(chaine));
11    }
12 }

```

Nous pourrions désormais utiliser ces méthodes comme si elles faisaient parties de la classe `String` :

```

1 string chaineNormale = "Bonjour à tous";
2 string chaineCryptee = chaineNormale.Crypte();
3 Console.WriteLine(chaineCryptee);
4 chaineNormale = chaineCryptee.Decrypte();
5 Console.WriteLine(chaineNormale);

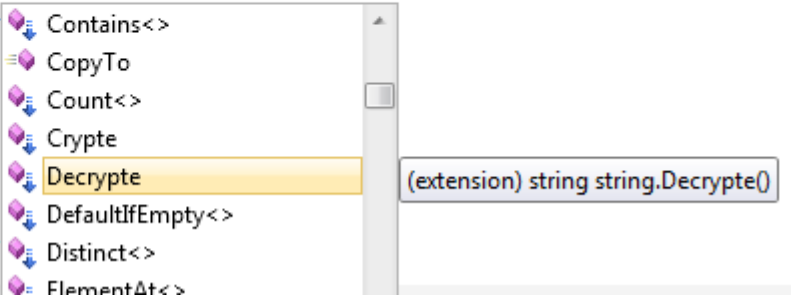
```

Pas mal non ? De plus, si nous regardons dans la complétion automatique, nous pourrions voir apparaître nos fameuses méthodes :

```

static void Main(string[] args)
{
    string chaineNormale = "Bonjour à tous";
    string chaineCryptee = chaineNormale.Crypte();
    Console.WriteLine(chaineCryptee);
    chaineNormale = chaineCryptee.;
    Console.WriteLine(chaineNormal
}

```



Plutôt pratique !

Limites : une méthode d'extension n'a pas accès aux méthodes privées ou variables membres internes à la classe. La preuve, les méthodes d'extensions sont **des méthodes statiques qui travaillent hors de toute instance de classe.**

Les méthodes d'extension doivent donc être statiques et situées à l'intérieur d'une classe statique.



Il faut faire attention à l'espace de nom où se situent nos méthodes d'extensions. Si le `using` correspondant n'est pas inclus, nous ne verrons pas les méthodes d'extension.

Méthode d'extension et interface

Les méthodes d'extension fonctionnent aussi **avec les interfaces**. Plus précisément, elles viennent **étendre toutes les classes qui implémentent une interface**.

Par exemple, avec deux classes implémentant l'interface `IVolant` :

```
1public interface IVolant
2{
3    void Voler();
4}
5
6public class Oiseau : IVolant
7{
8    public void Voler()
9    {
10        Console.WriteLine("Je vole");
11    }
12}
13
14public class Avion : IVolant
15{
16    public void Voler()
17    {
18        Console.WriteLine("Je vole");
19    }
20}
```

Si je crée une méthode d'extension prenant en paramètres un `IVolant`, préfixé par `this` :

```
1public static class Extensions
2{
3    public static void Atterrir(this IVolant volant)
4    {
5        Console.WriteLine("J'atterris");
6    }
7}
```

Je pourrais accéder à cette méthode pour les objets `Avion` et `Oiseau` :

```
1Avion a = new Avion();
2Oiseau b = new Oiseau();
3a.Atterrir();
4b.Atterrir();
```

À noter que le framework .NET se sert de ceci pour proposer un grand nombre de méthodes d'extension sur les objets implémentant `IEnumerable`, nous les étudierons un peu plus tard.

En résumé :

- Une méthode d'extension permet d'étendre une classe en lui rajoutant des méthodes.
- Il n'est pas recommandé d'utiliser des méthodes d'extension lorsqu'on dispose déjà du code source de la classe ou qu'on peut facilement en créer un type dérivé.
- On utilise le mot-clé `this` en premier paramètre d'une classe statique pour étendre une classe.

Application

1. Créer une méthode d'extension qui permet de compter le nombre de mots d'une chaîne. On considérera les séparateurs (espace et , ; ! ?).
2. Créer une méthode d'extension qui indique si un entier est plus grand qu'un autre entier.

Liens utiles

[Fonction de hachage](#)

[HashAlgorithm](#)